Block 4 – Session 1

Data Centers & Network Virtualization

Arquitectura de Xarxes

Universitat Pompeu Fabra

upf.

Outline

upf.

What Is the Cloud and How Is It Formed?

Every time you use Netflix, Gmail, or ChatGPT,
your request travels to a building full of servers.
What happens inside that building?

Learning objectives:

1. Understand why data centers exist and what they contain
2. Explain virtualization and distinguish VMs, hypervisors, and containers
3. Describe how networks are virtualized (vNICs, vSwitches, overlays)

upf.

Block 4 Roadmap

	Session 1 (today)	**Session 2**
Theme	*The infrastructure*	*The business model*
Topics	Data centers VMs & hypervisors VMs vs containers (overview) vNICs, vSwitches, vRouters Multi-tenancy & VLANs	Overlay networks (VXLAN) From virtualization to cloud NIST cloud definition IaaS / PaaS / SaaS VPC architecture Security groups & ACLs Governance & GDPR

Outline

upf.

What Is a Data Center?

- A **data center** is a facility housing computer systems and networking equipment
- Purpose: run applications and store data **reliably, at scale, 24/7**
- Ranges from small server rooms to **warehouse-scale** buildings

Who operates them?

- **Enterprises**: banks, hospitals, universities (their own IT)
- **Cloud providers**: AWS, Azure, GCP (sell capacity to others)
- **Colocation**: you rent space; provider handles power and cooling

Source: Barroso, Clidaras & Hölzle, *The Datacenter as a Computer*, 3rd ed., Morgan & Claypool, 2019.

A Data Center from the Inside

Rows of server racks, each containing dozens of servers, connected by structured cabling. Source: Cisco, *What Is a Data Center?*, cisco.com.

Inside a Data Center: Five Pillars

- **Compute**: CPUs that execute workloads (web servers, databases, analytics)
- **Accelerators**: GPUs and TPUs for AI/ML training and inference
  - The explosive growth of AI has made GPUs a critical data center resource
- **Storage**: SSDs and HDDs holding persistent data
- **Network**: switches, routers, and structured cabling connecting everything
- **Facilities**: redundant power supplies, cooling systems, physical security

The Problem with Physical Infrastructure

- Traditional model (1990s–early 2000s): **one server = one application**
- Typical server utilization: only **10–15%** of capacity
- Adding capacity means buying, racking, cabling → **weeks or months**
- Hardware failure takes down the entire service
- No way to scale down when demand drops

Key insight: most servers sit idle most of the time. We are paying for hardware we barely use.

upf.

The Cloud: From Waste to Efficiency

- **1990s–early 2000s**: one application per server; most capacity wasted
- **2001**: VMware makes it possible to run **multiple systems on one machine**
  - Technology: **Virtual Machines (VMs)**
- **Mid-2000s**: enterprises consolidate servers (60–80% utilization)
- **2006–2010**: Amazon, Google, and Microsoft start **renting computing capacity** online
  - Technology: **Cloud Computing** (AWS, Azure, GCP)
- **2010s–today**: lightweight alternatives emerge; automation at massive scale
  - Technologies: **Containers** (Docker) and **orchestration** (Kubernetes)

We will explore each of these technologies step by step throughout Blocks 4 and 5.

upf.

Scalability and Elasticity

- **Scalability**: ability to grow resources as demand increases
- **Elasticity**: ability to grow *and shrink* automatically
- Physical infrastructure provides **neither** in practice:
  - Cannot add servers in minutes
  - Cannot return servers when demand drops
- Virtualization enables both $\rightarrow$ foundation of **cloud computing** (Session 2)

upf.

Discussion: Data Centers

A company runs 50 servers, each at 10 percent utilization.
That is 50 machines doing the work of 5.
What would you do to reduce waste?

Hints: consolidation, fewer physical machines, higher utilization, isolation between workloads.

Outline

upf.

What Is Virtualization?

- **Virtualization** makes it possible for a single computer to act like many
- A software layer called **hypervisor** divides the physical resources (CPU, RAM, disk, NIC) into isolated **virtual machines (VMs)**
- Each VM gets its own share and believes it owns dedicated hardware

Why virtualize?

- **Resource efficiency**: consolidate workloads instead of one app per server
- **Cost savings**: fewer servers, less power, cooling, and space
- **Flexibility**: create or destroy environments in minutes
- **Isolation**: VMs are independent; failures do not propagate

upf.

Virtual Machines (VMs)

- A VM is an **isolated computing environment** with its own CPU, memory, network interface, and storage
- Runs a **full operating system** (Linux, Windows, etc.)
- Completely **isolated** from other VMs on the same host

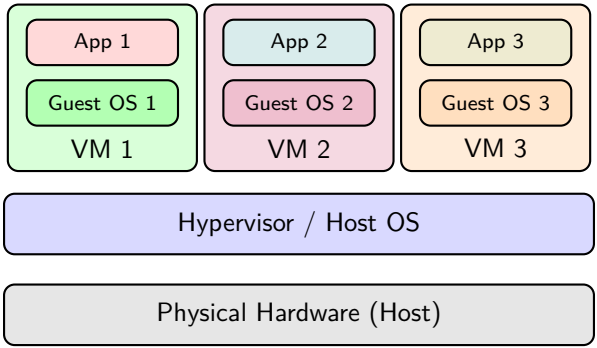
Key properties:

- **Encapsulation**: entire VM state stored as files → easy to copy, move, backup
- **Hardware independence**: the VM does not care about the underlying hardware
- **Snapshotting**: save and restore VM state at any point in time

Source: Popek & Goldberg, "Formal Requirements for Virtualizable Third Generation Architectures," *CACM*, 1974.

upf.

Host and Guest Systems



- **Host**: physical machine providing CPU, RAM, disk, NIC
- **Guest**: virtual machine running its own OS on the host
- Each guest believes it has **dedicated hardware** (abstraction)

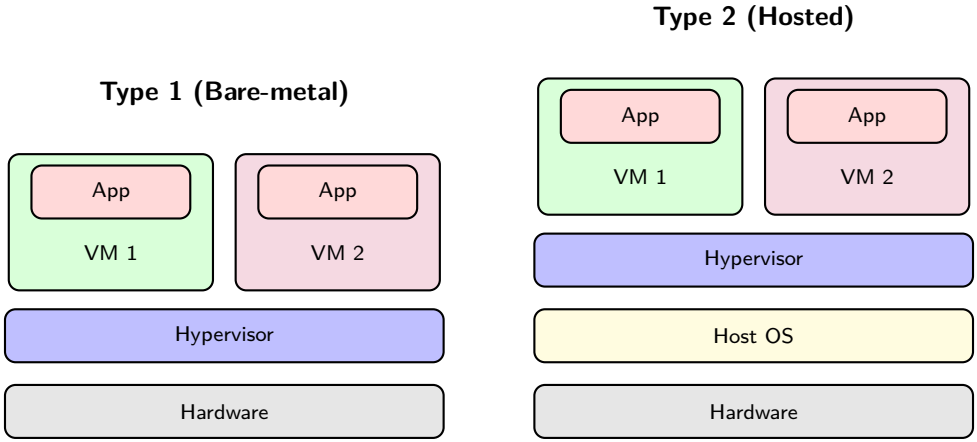
Hypervisors: The Key Enabler

- A **hypervisor** (or VMM) manages and allocates physical resources to VMs
- Two types:

	Type 1 (Bare-metal)	**Type 2 (Hosted)**
Runs on	Directly on hardware	On top of a host OS
Performance	Near-native	Some overhead
Use case	Data centers, production	Development, testing
Examples	VMware ESXi, KVM, Hyper-V	VirtualBox, VMware Workstation

Source: Barham et al., "Xen and the Art of Virtualization," *SOSP*, 2003.

Type 1 vs Type 2: Visual Comparison



- Type 1: hypervisor runs **directly on hardware** (no host OS) → less overhead, used in production
- Type 2: hypervisor runs **on top of a host OS** → easier to set up, used for dev/testing

VMs vs Containers: Overview

Virtual Machines

- Include a **full guest OS** per instance
  - A typical OS: 1–5 GB just for the OS itself
- Size: **GBs** (OS + app + dependencies)
- Boot time: **minutes**
- Strong isolation (hardware level)
- Can run **different operating systems**

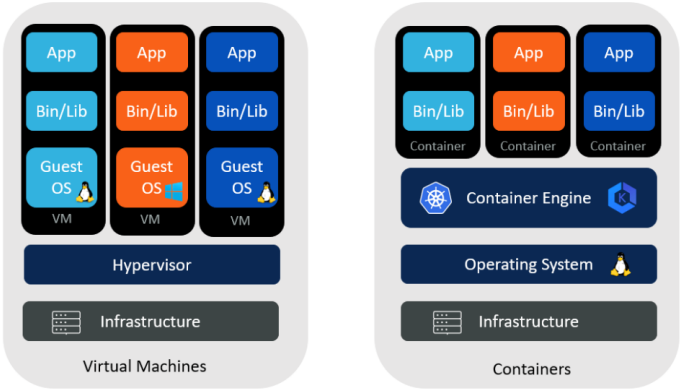
Containers

- **No guest OS**; share the host kernel
- Size: **MBs** (app + dependencies only)
- Boot time: **seconds**
- Good isolation (kernel namespaces/cgroups)
- All containers share the **same OS kernel**

This is just an overview. We will do a deep dive into container internals and networking in **Block 5**.

upf.

VMs vs Containers: Visual Comparison



Bins/Libs = binaries and libraries: the dependencies each app needs to run (e.g., Python, OpenSSL, libc).

Source: Aqueduct Technologies, "Containers and Virtual Machines," aqueducttech.com.

When to Use VMs vs Containers

- **Use VMs when** the app requires it:
  - Needs a **different OS** than the host (e.g., Windows app on a Linux host)
  - Requires **strong tenant isolation** (e.g., multi-tenant cloud)
  - **Legacy software** tied to a specific OS version
- **Use containers when** the app allows it:
  - Cloud-native or **microservices** applications
  - Need for **fast scaling** (seconds, not minutes)
  - **CI/CD pipelines**: build, test, deploy in identical environments
- **The trend**: most new applications are designed for containers
  - VMs remain essential for legacy workloads and strong isolation
  - Common pattern: containers run **inside VMs** (best of both worlds)

Microservices and container networking: **Block 5**.

upf.

Discussion: Virtualization

You need to deploy 200 instances of the same web application that must scale up and down every few minutes. Would you use VMs, containers, or both? Why?

Hints: boot time, resource overhead, isolation needs, how fast you need to scale.

Outline

upf.

From Physical to Virtual Networks

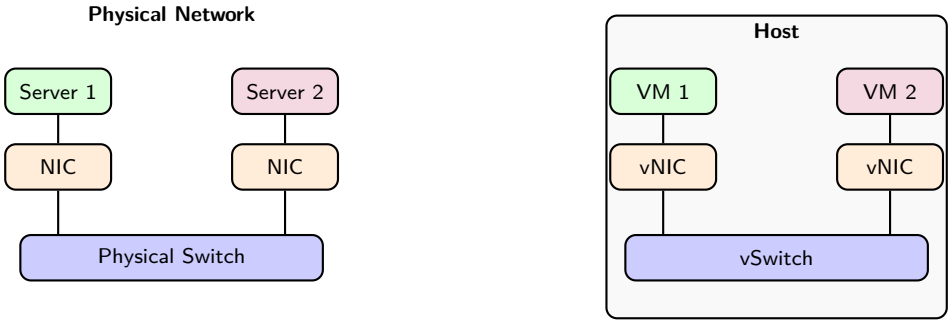
- You already know how physical networks work (Blocks 1 to 3):
  - NICs, switches, routers, VLANs, routing protocols
- When we virtualize servers, we also need to **virtualize the network**
- The same concepts apply, but implemented **in software** instead of hardware

Physical	**Virtual equivalent**
NIC (network interface card)	**vNIC** (virtual NIC)
Switch	**vSwitch** (virtual switch)
Router	**vRouter** (virtual router)
Cable	Internal software path

Key idea: if VMs think they have real hardware, they also need a **real-looking network**.

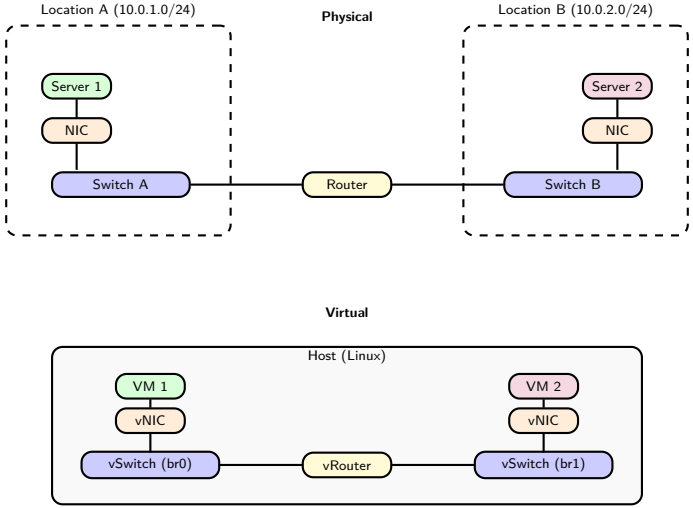
upf.

Physical vs Virtual: Side by Side (Same Network)



- Left: two physical servers connected by a physical switch and cables
- Right: two VMs connected by a virtual switch, all inside **one physical host**

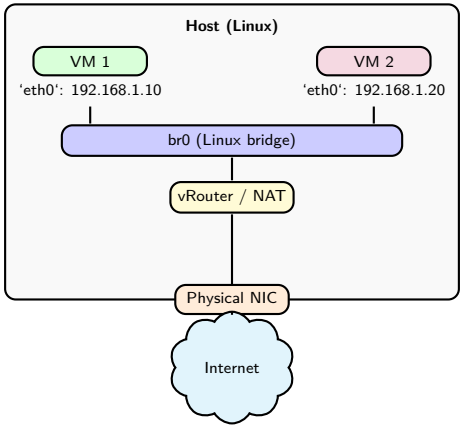
Physical vs Virtual: Side by Side (Different Networks)



Different subnets → router needed (L3). Physical uses hardware; virtual uses software inside one host.

Example: VM-to-VM Communication

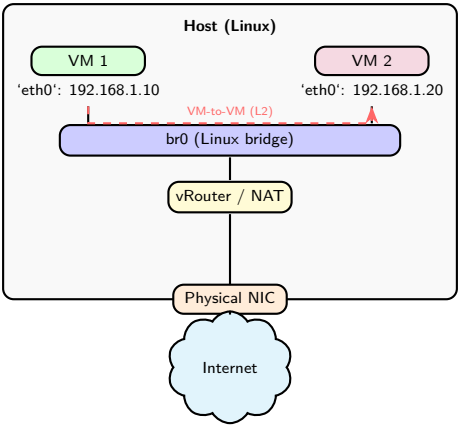
- Linux can act as a **Layer 2 switch** using a **bridge** (`br0`)
- VMs on the same bridge share a subnet: frames forwarded by MAC address



How does VM 1 reach VM 2? And how does it reach the Internet?

Example: VM-to-VM Communication (Solution)

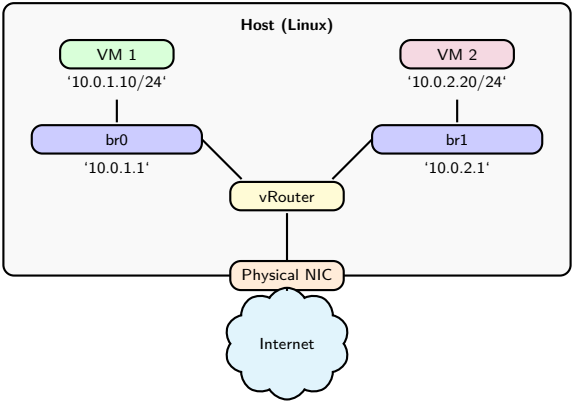
- Same subnet, same bridge → **L2 forwarding** (no routing needed)



Both VMs share `br0`: the bridge forwards frames by MAC address, just like a physical switch. No routing needed.

Example: VMs on Different Subnets

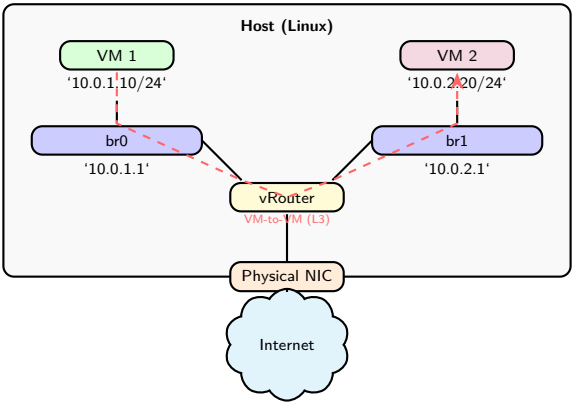
- Now VMs are on **different subnets**: they need L3 routing
- Two bridges (`br0`, `br1`), each with its own subnet



How does VM 1 (10.0.1.10) reach VM 2 (10.0.2.20)? What components are involved?

Example: VMs on Different Subnets (Solution)

- Different subnets → traffic must go through the **vRouter** (L3 forwarding)



Each VM has a default gateway pointing to its bridge IP. The host routes packets between subnets, just like a physical router.

Virtual Network Interfaces (vNICs)

- A **vNIC** is a software-emulated network interface card
- From the guest's perspective: behaves exactly like a real NIC
- Each VM gets **one or more vNICs**

Key characteristics:

- Has its own **MAC address** (software-assigned by the hypervisor)
- Has its own **IP address** configuration
- Connected to a **virtual switch** on the host
- Multiple vNICs per VM enable **multi-network** connectivity

Same MAC/IP concepts from Blocks 1 to 3, now virtualized inside the hypervisor.

upf.

Types of Virtual Switches

Type	Scope	Example
Standard vSwitch	Single host	VMware vSS, Linux bridge
Distributed vSwitch	Multiple hosts, central mgmt	VMware vDS, Hyper-V
SDN-based vSwitch	Programmable, software-defined	OVS, VMware NSX

- All types support **VLANs** (same 802.1Q from Block 3, now inside the hypervisor)
- OVS is open source and **programmable via software** → deep dive in **Block 5** (SDN)

Source: Pfaff et al., "The Design and Implementation of Open vSwitch," *NSDI*, 2015.

Bridging Modes

Three ways to connect a VM to the outside world:

Mode	VM visibility	External access?	Use case
Bridged	Same subnet as host	Yes (direct)	Production servers
NAT	Hidden behind host IP	Outbound only	Dev/testing
Host-only	Only sees the host	No	Isolated labs

- **Bridged**: VM appears directly on the physical network
- **NAT**: VM traffic translated through the host's IP (same NAT from Block 3)
- **Host-only**: completely isolated from external networks

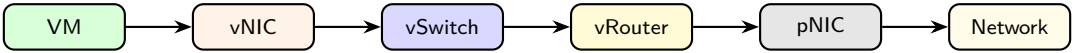
Virtual Routing and NAT

- A **virtual router** forwards traffic between virtual subnets
- Same function as a physical router, implemented **in software**
- Handles:
  - Routing between VMs on **different subnets**
  - **NAT** for VMs accessing external networks
  - Acting as **default gateway** for VMs

Same NAT concept from Block 3, now managed by the hypervisor instead of a physical device.

In Session 2 we will see how cloud providers offer NAT as a **managed service** (NAT Gateway).

Packet Flow: Virtual to Physical



1. VM generates packet with **virtual MAC/IP** as source
2. vNIC passes frame to **vSwitch** (L2 forwarding)
3. If destination is another subnet → **vRouter** (L3 forwarding)
4. vRouter may apply **NAT** (replace private IP with host IP)
5. Frame exits through **physical NIC** to the external network

Discussion: Virtual Networking

Two VMs on the same host want to communicate.
Does their traffic ever leave the physical machine?
What if they are on different subnets?

Hints: vSwitch handles same-subnet traffic locally; vRouter needed for different subnets; traffic may still stay inside the host.

Outline

upf.

Putting It All Together

- We have seen the **building blocks**: vSwitches, vRouters, vNICs, VLANs
- These components do not run in isolation; they are managed by **platforms**
- Platforms orchestrate VMs or containers and configure virtual networking under the hood

Two categories:

- **VM management platforms**: create and manage virtual machines at scale
- **Container orchestration platforms**: deploy and scale containerized applications

Cloud Management Platforms (VMs)

- VMs need a **management layer** to orchestrate resources
- These platforms let you build and manage your **own (private) cloud**

Platform	Description	Logo
OpenStack	Open-source, widely used in private clouds	openstack.
VMware vSphere	Enterprise leader, proprietary	vmware
Proxmox VE	Open-source, lightweight alternative	PROXMOX

As an alternative: **public cloud providers** (pay-per-use, no hardware to manage)

Provider	Logo
Amazon Web Services (AWS)	aws
Microsoft Azure	Azure
Google Cloud Platform (GCP)	Google Cloud

Container Orchestration Platforms

- Containers need **orchestration** to manage hundreds or thousands of instances
- Key tasks: scheduling, scaling, networking, self-healing

Platform	Description	Logo
Docker	Standard container runtime; package and run containers	
Kubernetes (K8s)	De facto standard for orchestration at scale	
OpenShift	Enterprise K8s distribution by Red Hat	

Discussion: Platforms

A company runs 50 legacy Windows applications and 200 microservices. Which platform(s) would you recommend and why?

Hints: VMs for legacy Windows apps (different OS); containers for microservices (fast scaling); possibly both on the same infrastructure.

Outline

upf.

Why Isolation Matters

- Data centers host workloads from **multiple tenants** (users, departments, companies)
- **Multi-tenancy**: shared infrastructure, but each tenant expects:
  - **Performance isolation**: my traffic is not affected by yours
  - **Security isolation**: you cannot see or access my data
  - **Address independence**: we can both use `10.0.0.0/24`

Without isolation: broadcast storms, ARP spoofing, IP conflicts, resource starvation.

upf.

VLANs in Virtual Environments

- Same **802.1Q** concept from Block 3, now applied to virtual switches
- Hypervisors assign VMs to specific VLANs via **vSwitch port groups**
- Traffic tagged with VLAN IDs: tenants separated at L2

Limitation: VLAN ID is 12 bits $\rightarrow$ max **4,096 VLANs**

Problem: 4,096 VLANs is **not enough** for a large cloud with thousands of tenants.

Source: IEEE 802.1Q-2022, "Bridges and Bridged Networks."

Micro-Segmentation

- Beyond VLANs: **per-VM or per-workload firewall policies**
- Goal: **least privilege**, each VM should only reach what it strictly needs
- Key concept in modern **zero trust** security architectures

In Session 2 we will see how cloud providers implement this with **Security Groups** (per-instance) and **Network ACLs** (per-subnet).

Discussion: Network Isolation

You manage a data center with 5,000 tenants.
Each tenant wants their own isolated network.
Can you use VLANs alone? Why or why not?

Hints: think about the VLAN ID limit (4,096), overlapping IP ranges, and what happens when VMs move between hosts.

Outline

upf.

Key Takeaways

1. **Data centers** house compute, accelerators (GPUs), storage, network, and facilities
2. Physical infrastructure is **rigid, underutilized, and slow to scale**
3. **Hypervisors** (Type 1 and 2) enable multiple VMs on one host
4. **Containers** are lighter than VMs but share the kernel (deep dive in Block 5)
5. Virtual networks mirror physical ones: **vNICs, vSwitches, vRouters**
6. **Multi-tenancy** requires isolation $\rightarrow$ VLANs and micro-segmentation

Next Session: Preview

In **Session 2** we build on today's foundation:

- **Overlay networks** (VXLAN): scaling beyond the 4,096 VLAN limit
- Virtualization is the technology; cloud is the **business model**
- NIST definition and **5 essential characteristics**
- Service models: **IaaS / PaaS / SaaS**
- **VPC** architecture: subnets, route tables, gateways
- Cloud security: **Security Groups + Network ACLs**
- Governance, **data sovereignty**, GDPR

upf.

Discussion

A company runs 200 VMs on 10 physical servers.
Each team wants an isolated network.
What virtual networking components do you need
and how would you connect them?

Hints: think about vNICs, vSwitches, VLANs, and routing between subnets.

References

1. Popek & Goldberg, "Formal Requirements for Virtualizable Third Generation Architectures," *CACM*, vol. 17, no. 7, 1974.
2. Barham et al., "Xen and the Art of Virtualization," *SOSP*, 2003.
3. VMware, "Understanding Full Virtualization, Paravirtualization, and Hardware Assist," 2007.
4. Pfaff et al., "The Design and Implementation of Open vSwitch," *NSDI*, 2015.
5. IEEE 802.1Q-2022, "Bridges and Bridged Networks."
6. Barroso, Clidaras & Hölzle, *The Datacenter as a Computer*, 3rd ed., Morgan & Claypool, 2019.
7. NIST SP 800-125, "Guide to Security for Full Virtualization Technologies," 2011.
8. Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021.
9. Red Hat, "What is a Virtual Machine?", redhat.com/en/topics/virtualization/what-is-a-virtual-machine.
10. Red Hat, "Containers vs VMs," redhat.com/en/topics/containers/containers-vs-vms.
11. Cisco, "What Is a Data Center?", cisco.com/site/us/en/learn/topics/computing/what-is-a-data-center.html.